

Computer security

Indice

Intro	2
Cryptography	3
Definitions	4
The perfect cipher (and why it's useless)	4
Cryptographically safe PRNGs	5
Random functions and permutations (block ciphers)	5
Block cipher modes	6
Data integrity	6
Cryptographic hashes	6
Asymmetric encryption	7
Diffie-Hellman key agreement	7
Public key encryption	7
Authenticating data	8
Digital certificates	8
Fundamentals of information theory	8
Authentication	9
To-know factors (passwords and pins)	9
To have factors (tokens, smart cards and smartphones)	10
To be factor (biometric)	10
Single Sign On	11
x86 crash course	11
Architecture	11
Assembly	11
64-bit extension (AMD64 or x86_64)	12
Program layout	12
Calling functions	13
Access control	14
Discretionary access control	15
Mandatory Access Control	15
Software security	16
Binary exploitation	16
Buffer overflows	16
nop sled	17
Shellcode	17
Using environment variables	18
Using built-in functions	18
Other ways of jumping	19
Defending against buffer overflows	19
Format string bugs	19

What to edit	20
Countermeasures	20
Web applications	21
Cross-site scripting (XSS) and Same-Origin Policy	21
Content Security Policy	22
SQL injection	22
Recap on code injection problems	22
Information leaks	22
Password security	23
Bruteforcing protection	23
Cookies	23
Cross-site request forgery (CSRF)	23
Network protocol attacks	23
DoS	24
Sniffing	24
Spoofing	24
ARP Spoofing (or Cache poisoning)	24
Filling the CAM tables	25
IP address spoofing	25
Man In The Middle (MITM)	25
DNS cache poisoning	25
DHCP poisoning	26
ICMP and ICMP redirects	26
Route mangling	26
Secure network architectures	26
Firewalls	26
Packet filters	26
Application proxies	27
Architecture of secure networks	27
Multi-zoned networks	27
VPNs	28
SSL, TLS and SET	28
SSL/TLS	28
SET	29
Malicious software	30
Viruses	30
Worms	31
Botnets	31
Anti-malware	31
Virus stealth techniques	32

Intro

Security **always has an adversarial component**: there is someone that wants to abuse the system.

A vulnerable system is a system where exists a shortcut that is not known to the system designer.

If the system secured is a computer, **for it to be secure we need it to have 3 main properties (CIA)**:

1. **Confidentiality**: information should be accessed only from authorized people
2. **Integrity**: information should be modified only by persons authorized to do so
3. **Availability**: information should be available to all authorized parties within specified time constraints

Providing these three elements is an engineering problem.

The **availability constraint is the business requirement** and the most difficult one to keep satisfied: data is always more and to be made more readily available.

A vulnerability is something that allows us to violate one of the constraint. On the other hand, **exploits are a specific way to use one or more vulnerability to accomplish a specific objective that violates the constraint.**

In the case of a vulnerable lock: the vulnerability is friction, the exploit is using tension wrench and pick to pick the lock

In computers, there are scenarios in which we can change the vulnerable code but there are also cases in which we cannot due to various reason (cost, legacy compatibility etc).

The security of a system is not correlated to the level of protection of said system. We can have protected system be less secure than unprotected ones. Even for protection, **we need to define what threats we protect from.** When we are talking about security, **we always need to define the threat model, otherwise our discussion is useless.**

Assets are what is valuable for an organization. Since we are talking about computer security, we are talking about IT assets. **Most of the time, it is not the computer that is valuable, but what they do (for example the data they handle or the industrial process they manage).** Even reputation can be an asset.

A threat is a circumstance potentially causing a CIA violation.

An attack is an intentional use of one or more exploits with the objective of compromising a system's CIA. Threat agent is someone/something that makes an attack happen.

In general **risk is the statistical and economical evaluation of the exposure to damage because of the presence of vulnerabilities and threats.**

$$\text{Risk} = \text{Asset} \times \text{Vulnerabilities} \times \text{Threats}$$

Assets and vulnerabilities are controllable, while threats are not. Security is the balance of **reduction of vulnerabilities + damage containment vs cost.**

The **cost of security is both direct** (management, operational and equipment) **and indirect** (less usability, lower performance and reduced productivity). A **fallacy is changing the system to reduce privacy to increase security.** This paradigm is **always a net loss for the system** since we are **exchanging the confidentiality of users** (confidentiality) with a **small increase of security of the system.**

Another problem is that, often, **security measures impose a cost on the users but do not benefit them.** This means that **if users do not see value in the measures, they are going to go out of their way to avoid them.**

In security, **throwing money around does not solve problems: the best way to increase security is often to do nothing than to do something badly.** Once a measure has been introduced, however, **no one would bear the cost of the possibility (even low) of an attack made possible by said removal.** Thus once security measures are made permanent **they are very rarely reverted.**

A part of the system can be assumed to be safe (meaning that adversary can be assumed to not tamper): these are the **trusted elements.** We need to assume the safety of some components to avoid the **chicken-and-egg problem.**

Cryptography

It is the study of techniques to allow secure communication and data storage in presence of attackers. It provides:

1. **Confidentiality**
2. **Integrity/freshness:** detect/prevent tampering
3. **Authenticity:** data and their origin are guaranteed

4. **Non-repudiation:** data creator cannot repudiate created data
5. Advanced features: **proof of knowledge/computation**

Note: when in cryptography we talk about the **attacker**, we are **assuming their capabilities, not whether said attacker is feasible or not**.

A side-channel is a characteristic that betrays a secret we are trying to hide.

We say that an encryption method provides **forward-secrecy** if **compromising the key does not compromise all messages that came before**.

One of the **most important features** of a crypto algorithm is that **knowledge of said algorithm does not compromise it**. The first instance separation between key and algorithm comes from Bellaso (1553).

Kerchoff's 6 principles for a good cypher:

1. **It must be practically, if not mathematically, unbreakable**
2. **It should be possible to make it public, even to the enemy**
3. **The key must be communicable without written notes and changeable whenever the correspondents want**
4. It must be applicable to telegraphic communication (nowadays it is not very applicable)
5. It must be portable, and should be operable by a single person
6. Finally, given the operating environment, it should be easy to use, it should not impose excessive mental load, nor require a large set of rules to be known

Requirement number 2 it is the most basic requirement for modern cryptography: the only thing that we need to keep safe, is the key.

Shannon, in 1949, **proved the existence of mathematically unbreakable algorithms**, even though unpractical. **Nash**, in 1955, elaborated by **proving that the effort needed to break a computationally intensive algorithm with a long enough key is insurmountable**.

Definitions

- **Plaintext space $\mathbf{P}$:** set of possible messages
 - $\{0, 1\}^l$
- **Ciphertext space $\mathbf{C}$:** set of possible ciphertexts
 - $\{0, 1\}^{l'}$, not necessarily $l = l'$
- **Key space $\mathbf{K}$:** set of possible keys
 - $\{0, 1\}^\lambda$
- **Encryption function $\mathbb{E} : \mathbf{P} \times \mathbf{K} \rightarrow \mathbf{C}$**
- **Decryption function $\mathbb{D} : \mathbf{C} \times \mathbf{K} \rightarrow \mathbf{P}$**

Our design goal is to **make sure that if someone doesn't know the key, it cannot access the plaintext**.

However, this is a simplistic view since the attacker is considered passive. **In reality attackers can be smarter:**

- **Eavesdropper (ciphertexts only attack)**
- Knows a set of possible plaintexts (**known plaintext attack**)
- Active attacker may tamper with the data and observe the reactions of a decryption-capable entity (**chosen plaintext attack**)

The perfect cipher (and why it's useless)

In a **perfect cipher**, for all plaintexts and ciphertexts:

$$P(\text{plain sent is our plaintext}) = P(\text{plain sent is our plaintext} | \text{cipher sent is the cipher})$$

In other words: **seeing a ciphertexts gives us no information on what the plaintext corresponding to it could be**.

Theorem: any symmetric cipher with $\mathbf{P} = \mathbf{K} = \mathbf{C}$ is perfectly secure iff:

1. every key is used with equal probability
2. a unique key maps a given plaintext into a given ciphertext.

Basically this theorem gives us a **perfect (but useless) cipher**.

A **simple algorithm** needs a **fresh perfectly random key** for every message of length equal to the message. We can simply **XOR the message with the key to encrypt**, and **XOR the ciphertext with the key to decrypt it**. The algorithm is obviously impractical due to the length of the key and also the need to extract a fresh key for every message.

A more **practical assumption** is to try to achieve not mathematic perfection, but to make our cipher **practically unbreakable**. This means we need to make our attacker solve a problem that is **exponentially complex**.

An outline for proving the computational security of problem is:

1. Define the ideal attacker
2. Assume a given computational problem is hard
3. Prove that any non-ideal attacker solves the hard problems

An **attacker** is represented as a program able to call given libraries, libraries implement the cipher at hand. We define the **security property** as answering to a given question. The **attacker wins the game** if it breaks the security property more often than what is possible through a random guess.

Cryptographically safe PRNGs

A **CSPRNG** is a deterministic function $\{0,1\}^\lambda \rightarrow \{0,1\}^{\lambda+l}$ whose output cannot be distinguished from a uniform random sampling of $\{0,1\}^{\lambda+l}$ in $\text{poly}(\lambda)$. l is the generator's stretch.

In practice, we have only **candidate CSPRNGs**: we do not have proof that such a function exists; moreover proving that a CSPRNG exists implies $P \neq NP$. Practically CSPRNG are built with Pseudorandom Permutations (PRPs), which themselves are built from Pseudorandom functions.

Random functions and permutations (block ciphers)

Consider the set of all functions $= \{ \{0,1\}^{in} \rightarrow \{0,1\}^{out} \}$. A uniformly randomly sampled $f \leftarrow^{\$} \mathbf{F}$ can be encoded by a 2^{in} entries **table**, each entry *out* bit wide. This means that $|\mathbf{F}| = (2^{out})^{2^{in}}$.

A **pseudorandom function** is a function prf_{seed} taking an input and a λ bit seed. The **entire function** is described by the value of the seed. It cannot be told apart from a random function extracted from $\mathbf{F}$ in $\text{poly}(\lambda)$.

A **pseudorandom permutation** is a **bijective PRF**. It is:

1. Uniquely identified by the value of the seed
2. It is **not possible to tell apart** in $\text{poly}(\lambda)$ from a **random function**
3. It is a **permutation of all the possible** $\{0,1\}^{len}$ strings

Operatively speaking, it acts on a block of bits and outputs another of the same size. The output “looks unrelated” to the input. Its actions is fully identified by the seed.

No formally proven PRP exists (for the same reasons as CSPRNGs). A typical construction is starting with a small bijective boolean function f and sequentially computing f until we are satisfied.

Real world PRPs go by the **historical name of block cyphers**. They are considered broken if with less than 2^λ operations, we can tell them apart from a PRP, e.g. via deriving the input corresponding to an output without the key. The key length λ is chosen to be large enough so that computing 2^λ guesses is not practically feasible.

Some famous block ciphers are:

- **AES**: most common block cypher
- **DES**: broken, the key is too short. A patch is to apply the algorithm three times

Block cipher modes

Block ciphers can be used in a variety of different encryption modes:

1. **ECB** (Electronic code book) consists of **applying the cipher with the same key to each block independently**. The biggest flaw of this method is that **if two blocks are identical, they will be equal even when encrypted**. Furthermore, it has very weak integrity protection since the cipher cannot tell if any block has been altered.
2. **Counter (CTR)** mode consists of **feeding a counter to each block cipher**. **The output of the cipher is then XORed with the plaintext** to obtain the ciphertext for that block. This mode, however, is **not effective against chosen plaintext attacks**: the encryption is **deterministic** (the counter always starts at 0).
3. **CPA-secure CTR** is like the regular CTR, however **for each encryption a NONCE (one time use number) is used as the counter start**. To enable decryption, **the nonce needs to be public**.
4. Symmetric **ratcheting** is a technique that **makes impossible to roll-back the encryption procedure**.

A problem that all these schemes suffer is that of **malleability: making changes to the ciphertext maps to predictable changes in the plaintext**. To avoid this issue we can: create a non-malleable scheme (not easy) or **add a mechanism for validating data integrity**.

Data integrity

To provide integrity, we can **add a small piece of information (tag) that allows us to test for the message integrity of the encrypted message itself**. Adding it to the plaintext and then encrypting it is not a good idea. These tags are called **MACs** (Message Authentication Codes) (name is very misleading: they do not authenticate anything).

A MAC is comprised of a **pair of functions**:

1. `compute_tag(string, secret_key) -> tag`: returns the tag for the input string
2. `verify_tag(string, tag, secret_key) -> bool`: returns whether the tag is valid

The ideal attacker **knows as many message-tag pairs as he wants**. However **he cannot forge a valid tag** (including splicing other messages' tags) **for a message for which he does not know the tag already**.

We can **build a MAC using a block cipher in the CipherBlock Chaining mode**:

1. **For each block we XOR the plaintext block with the previous block's result**
 - All ciphers use the same secret key
2. For the first block we use 0 as the init

CBC-MAC is secure for prefix-free messages. If we encrypt the tag once more we solve this issue.

Testing the integrity of a file requires, however, to compare it bit by bit with an intact copy or read it entirely to compute a MAC. It would be nice to have a short fixed length string independent of the file size. However, due to the mismatch of cardinality between the total number of files and the total number of strings of length k .

Cryptographic hashes

A cryptographic hash is a **function** $H : \{0, 1\}^* \rightarrow \{0, 1\}^l$ for which the **following problems are hard**:

1. Given $d = H(s)$ find s (**first preimage collision resistance**)
 - Takes $\mathcal{O}(2^l)$ hashes computations to guess s
2. Given $s, d = H(s)$ find $r \neq s : H(r) = d$ (**second preimage collision resistance**)
 - Takes $\mathcal{O}(2^l)$ hash computations to guess r
3. find r, s with $r \neq s$ and $H(s) = H(r)$ (**collision**)
 - Takes $\approx \mathcal{O}(2^{\frac{l}{2}})$ hash computations

1st preimage collision implies the possibility of 2nd preimage collisions, however the opposite is not true.

The output bitstring of a hash is known as a digest.

Some uses for hashes:

1. Store/compare hashes instead of values.
2. Building MACs: generate tag hashing together with the message and a secret; verify tag recomputing the same hash.
 - See HMAC
3. Forensic uses.

Asymmetric encryption

Agreeing on a secret has to be done on a trusted channel. Is there a way to **share a key on a public channel, without compromising the message**? Yes: asymmetric encryption.

Up to now, enumeration of the secret parameter was the best possible attack. **Asymmetric cryptosystems rely on hard problems for which bruteforcing the secret parameter is not the best attack.**

This means that **comparing bit-sizes of the secret parameters between algorithms instead of the actual complexities is very wrong** for this type of encryption, do this only for algorithms of the same type. God forbid you do this with symmetric algorithms.

Diffie-Hellman key agreement

The goal is to make two parties share secret values with only public messages. **The attacker can:**

1. **Eavesdrop anything, but not tamper**
2. This **assumptions** should hold:
 - Let $(G, \cdot)$ a group of finite prime order q and on a generator g of this group. Let there be two numbers a, b sampled uniformly from $\{0, \dots, |G| - 1\}$. λ will be the length in bits of $\text{len}(a) \approx \log_2 |G|$.
Given g^a and g^b , finding g^{ab} costs more than $\text{poly}(\log |G|)$.
 g and λ are public.

The assumptions basically requires an attacker to be able to solve the discrete logarithm problem in polynomial time. Usually the chosen group is subgroup of the natural numbers of order p (still prime and appropriately large). The group can also be defined by an elliptic curve.

The agreement goes like this:

1. Alice picks a and sends g^a to Bob
2. Bob picks b and sends g^b to Alice
3. Each party computes $(g^b)^a$ and $(g^a)^b$ respectively. Thanks to the commutativity of the group, these two are equal. Now we have our key.

Public key encryption

Based on the same ideas of the DH key agreement, we can define asymmetric key encryption: the **key-pair generation algorithm** generates two keys, one **public** and one **private**. The **public part** can be shared publicly and is used for **encrypting a plaintext** that can only be decrypted with the associated private key.

It is **computationally hard** to:

1. **Decrypt a ciphertext without the private key**
2. **Compute the private key given the public key**

Asymmetric algorithms are **very slow**, so implementing it like this in the real world is not really feasible for fast communication. **What is done, instead, is called key encapsulation:**

1. **Alice chooses a large random secret for a pre-agreed symmetric cipher, encrypts it using Bobs public key and sends it.**
2. **Bob receives the message, decrypts it and communication continues using the shared secret on a symmetrically encrypted channel.**

This is basically what TLS and the like do.

Authenticating data

To build a secure hybrid encryption scheme **we need to be sure that the public key the sender uses is the one of the recipient**. We'd like to be able to verify the authenticity of a piece of data without a pre-shared secret.

Digital signatures are used for this: they are asymmetric cryptographic algorithms. Signatures cannot be repudiated by the user.

It uses the **same principles as public key encryption, however it flips it**: we **sign** the message with our **private key**, meaning that **everyone with access to our private key can verify it**.

Due to the slowness of asymmetric encryption and due to space constraints, we **sign only a hash of the message and append it to the message**.

Signing documents **makes sense only if the document is completely static**. If it is not, we are signing only the program that generates the content, not the content.

Digital certificates

We have one more problem: **how can we be sure that the public key corresponds to the correct identity (public key binding problem)?**

A **PKI** is a **system that allows people to bind a certain public key with an identity**. A **digital certificate** is **basically a file that contains the name of the entity and a public key**. The most used standard for digital certificates is X.509v3.

Digital certificates are the best way to **sign documents**, since we solve the binding problem.

Digital certificates need to be themselves signed, however who signs them? A **certificate authority** is a **trusted issuer that signs certificates**. But **who signs the certificate authority's certificate?** **Another CA**. This creates a **signing tree**, however it introduces an infinite recursion. This means **we need to have a root CA that is self-signed, trusted a-priori and whose certificate is stored in trusted storage**.

The root CA is not a certificate anymore, it is a statement of authority: we cannot verify it, we need to trust it.

Fundamentals of information theory

From information theory we need mainly 2 things: qualitatively frame “luck” and “guessing”.

Basics definitions:

1. Communications takes place between two endpoints:
 - The sender, made of an information source and an encoder
 - The receiver, made of an information destination and an encoder.
2. Information is carried by the channel in the form of a sequence of symbols of a finite alphabet.

The **receiver gets information only through the channel: it will be uncertain on what the next symbol is**. Acquiring information is **modeled through a random variable $\mathcal{X}$** : the **closer the variable is to a uniform distribution, the higher the amount of information I get from knowing the outcome**. Encoding maps each outcome as a finite sequence of symbols.

Entropy is basically a **measurement of uncertainty**. The desired properties are: non-negativity, “combining uncertainties” should map to adding entropies.

Entropy definition: Let $\mathcal{X}$ be a d.r.v. with n outcomes in $\{x_0, \dots, x_{n-1}\}$ with $P(\mathcal{X} = x_i) = p_i$ for all i . The entropy of $\mathcal{X}$ is $H(\mathcal{X}) = \sum_{i=0}^{n-1} -p_i \log_b(p_i)$.

The measurement unit of entropy depends on the base of the logarithm. The typical case for $b = 2$ is bits.

Shannon's noiseless coding theorem (informal): It is possible to encode the n outcomes of i.i.d. random variables, each one with entropy $H(\mathcal{X})$ into no less than $nH(\mathcal{X})$ bits per outcome. If less bits are used, some information is lost.

Min-entropy: we define the min-entropy of $\mathcal{X}$ as $H_\infty(\mathcal{X}) = -\log(\max_i p_i)$.

Intuitively, the **min-entropy** is a uniform distribution where the probability of each outcome is $\max_i p_i$. This means that **guessing the most common outcome of $\mathcal{X}$ is at least as hard as guessing a $H_\infty(\mathcal{X})$ bit long string.**

Authentication

An important distinction is that **between identification and authentication:**

1. In **identification** an entity declares an identifier
2. In **authentications** an entity provides proof that verifies its identity

Authentication can be **unidirectional or bidirectional** and can happen between any entity. **Authentication is subdivided in 3 factors:**

1. Something that the entity knows (**to know**)
2. Something that the entity has (**to have**)
3. Something that the entity is (**to be**)

Multi-factor authentication uses two or three of these factors.

To-know factors (passwords and pins)

Advantages:

- **Low cost**
- **Ease of deployment**
- **Low technical barrier**

Note: this advantages are **mainly for the implementer** of the authentication method and not the user.

Disadvantages (passwords can be:)	Countermeasures (enforce passwords that)
Stolen/snooped	Change/expire
Guessed	Are long and have rich character sets
Cracked (enumerated)	Are not related to the user

These **countermeasures are costs** because **humans are not machines**: we are unable to keep secrets and have trouble remembering arbitrary strings for a long time. This means that **we can't apply all the countermeasures, we need to choose based on the most likely attack we want to defend.**

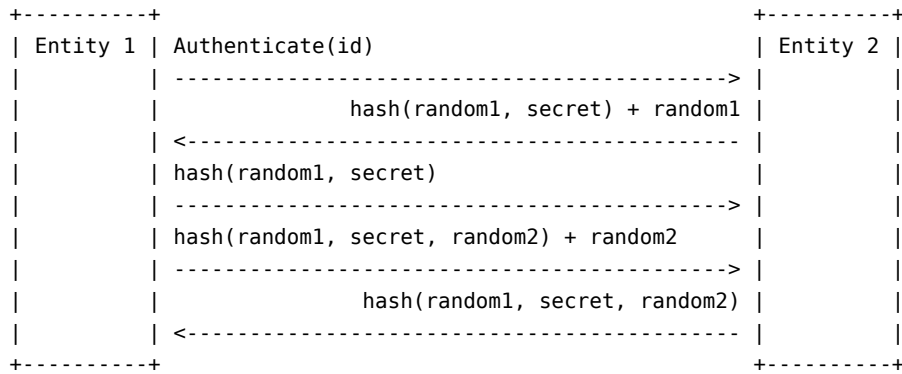
Countermeasure	Snooping	Cracking	Guessing
Complexity	Not effective	Very effective	Effective
Change	Very effective	Effective	Effective
Relation to user	Not effective	Not effective	Very effective

The **best countermeasure is user education**: the human is the weak link. We need to enforce strong passwords, enforce expiration policies, use password meters to balance usability.

Authentication is about sharing a secret. **How do we minimize the risk that secrets get stolen?**

1. Use **mutual authentication** if possible
2. **Challenge-response scheme**: we find a problem that can be solved only if you have the secret and only communicate the solution to said problem.

A sketch of a challenge-response scheme is the following:



Random data is used to avoid replay attacks

On the entity part, we need a way to **safely store passwords without compromising confidentiality**:

1. **Cryptographic protection**: never store passwords in clear (**hashing + salting**)
2. **Access control** policies
3. **Never disclose secrets in password-recovery schemes**

We also need to be careful of the **caching problem**: information could be held in intermediate storage locations.

To have factors (tokens, smart cards and smartphones)

The user must prove that it possesses something.

Advantages:

- **Human factor** (less likely to hand out a key)
- **Relatively low cost**
- **Good level of security**

Disadvantages	Countermeasures
Hard to deploy correctly	None
Can be lost or stolen	Use with second factor

This factor **works best in conjunction with another factor**.

Some classic technologies that implement this factor are:

- One time password generators: they are simple machines that contain a secret key that computes a MAC of a counter synchronized with the host.
- Smart cards: a CPU with non-volatile ram containing a private key. The smart card authenticates itself with a challenge-response. It uses the private key to sign the challenge. Should be tamper-proof.
- TOTP: software that implements the same functionality as password authenticators (see Google Authenticator). Only big difference: TOTPs run on general purpose systems, so they are more difficult to secure

It is really important that the key the device holds never leaves the device. If the key leaves it, the factor degrades to a “to know” factor.

Note: **not all devices are equally strong**. Each has its own weaknesses (e.g. see SIM-swapping for SMS 2FA).

To be factor (biometric)

User must prove that it has some specific characteristics.

Advantages:

- **High level of security**
- **Requires no extra hardware to carry around**

Disadvantages	Countermeasures
Hard to deploy	None
Probabilistic matching	None
Invasive measurements	None
Can be cloned	None
Biological characteristics can change	Frequent re-measurement
Privacy sensitivity	Secure the enrollment process
Users with disabilities	Use a (maybe weaker) alternative

Single Sign On

SSO tries to solve the problem of managing and remembering multiple passwords. The solution is to have **only 1 identity** (therefore 1 password) with possibly 2FA on a **single trusted host**. This means that **hosts authenticate only on this trusted host, other hosts can ask the trusted element if the user is authenticated**.

Examples of this flow are Shibboleth (AunicaLogin) and **OAuth**.

The problem of the method is the **reliance on a single point of trust**: if it is compromised, the whole thing comes crashing. Moreover, **the password reset scheme must be bullet proof since the email becomes a trusted element**.

SSO is a complex flow that is difficult to get right. Libraries exist, but they can contain bugs.

x86 crash course

Architecture

x86 assumes the Von Neumann architecture. It uses a little-endian endianness. The architecture has several registers, the most important are:

1. General purpose:
 - EAX, EBX, ECX, EDX
 - ESI, EDI (source and destination for string op)
 - EBP (base pointer)
 - ESP (stack pointer)
2. Instruction pointer EIP
 - We do not have access to it directly. It can only be modified by control instructions like `jmp`, `call`, and `ret`
 - It can be read from the stack
3. Program status and control: EFLAGS

If we remove the E from the general purpose registers, we can access them in 16-bit compatibility mode (lower half). Inside this half we can further split them into high and low by substituting the X with H and L respectively.

The EFLAGS register is a 32-bits register of boolean flags. It contains flags such as:

- Program status: overflow, sign, zero, auxiliary carry (BCD), parity, carry
- Program control: direction flag (controls string instructions)
- System: control operating-system operations

All instructions work on: bytes, words (2 bytes), dwords (4 bytes) and qword (8 bytes).

Assembly

x86 assembly has two main syntaxes: intel (default on windows) and AT&T (default in most UNIX tools). Beware, the order of operands IS different. We will use the Intel syntax since it is more readable.

In x86 instructions have variable length. They are formatted as such:

1. Prefix + OPCODE: 1 to 3 bytes
 - Note: the same instruction may have different OPCODES depending on how it is called
2. Operands

Instructions:

1. Data transfer:
 - `mov dst src`: moves the value from a source to a destination:
 - `mov eax, 4h`: moves a constant into `eax`
 - `mov eax, ebx`: moves from register to register
 - `mov eax, [ebx + 4h]`: moves from address in `ebx + 4` to `eax` (we can use arithmetic expressions into the indirect addressing block, e.g. `[edx + ebx*4 + 8]`)
 - Memory to memory is invalid
 - `lea dst src`: like `mov`, but it stores the address, not the value. It does not access memory.
2. Integer arithmetic:
 - `add dst src`: source can be any addressing type, while destination only register and memory.
 - `sub dst src`, `neg val`, `and`, `or`, `xor`, `not` work at the same way
 - `mul src`: unsigned multiply (for signed we use `imul`). The implied operands are based on the size of the source:
 - `AL, AX, EAX` as first operand
 - `AX, DX:AX, EDX:EAX` as destination
 - `R1:R2` mean that high bits will be in `R1` while the lower ones in `R2`.
 - `div src`: computes the quotient and remainder. The implied operand is `*DX:*AX`. The result is stored into `EAX`, the remainder into `EDX`. Signed division is `idiv`.
 - `cmp op1, op2`: sets the correct flags in `EFLAGS` after `op1 - op2`
 - `test op1, op2`: sets the correct flags in `EFLAGS` after `op1 & op2`
3. Control flow:
 - `jmp addr`: jumps to the given address or to an offset
 - `j<cc>`: conditional jump. The condition is dictated by `cc` (e.g. `jz`, `jlt`)
 - `nop`: the OPCODE is `0x90`
4. Interrupts and syscalls:
 - `int val`: generate a software interrupt with number `[0; 255]`. These are OS dependent (e.g. `80h` are Linux syscalls)
 - `syscall`: used for calling syscalls on Linux 64 bit
 - `sysenter`: the same thing as `syscall`, but for Windows

64-bit extension (AMD64 or x86_64)

Mainly it added 8 new general purpose (`R8` to `R15`) registers and expanded the previous registers to 64 bit (they use the `R` prefix, e.g. `RAX`, `RBX`...).

Program layout

We have two main binary formats:

1. PE (Portable Executable): used by Windows
2. ELF: common on other OSs (except MacOS)

In both cases we are more interested in which way the executable is mapped into memory.

In ELF files we have different sections (PEs have similar sections):

- `.plt`: stubs for external linking
- `.text`: executable instructions
- `.rodata`: read-only data
- `.data`: initialized data
- `.bss`: uninitialized data. It is initialized with all 0s at startup
- `.debug`: symbols for debugging
- `.got`: global offset table

Low addresses (0x80000000)

```
+-----+
| Shared libraries |
+-----+
| .text           |
+-----+
| .bss            |
+-----+
| Heap            |
+-----+
|                 |
|                 |
+-----+
| Stack           |
+-----+
| env             |
+-----+
| argv            |
+-----+
```

High addresses (0xbfffffff)

Calling functions

We can manipulate the stack pointer using the `push src` instruction (`val` is an immediate or register).

```
push eax
; equivalent to
sub esp, 4
mov DWORD PTR [esp], eax
```

The `pop dst` instruction does the opposite: loads into the destination a word off the top of the stack and moves `esp` accordingly.

```
pop eax
; equivalent to
mov eax, DWORD PTR [esp]
add esp, 4
```

To call a function, we use `call func`: it pushes to the stack the address of the next instruction and moves the `EIP` to the first instruction of the routine. `ret`, which does the exact opposite as `call` (i.e. reads the popped return address from the stack and updates the `EIP`). Calling a function, however, requires the following operations:

- A stack frame is the area allocated to a function
- `EBP` register is a pointer to the beginning of a function's frame
- At the beginning of a function we need to:
 - Save the old `EBP`
 - Set `EBP` to the address of the function's frame
 - The `EBP` can be used to access local variables (pushed on top of the stack) or the function arguments (stored under the `EBP` and under the return address)

Calling functions requires that caller and callee agree on which registers are preserved, where arguments are stored and where the return value is stored (calling conventions). This is part of the ABI (Application Binary

Interface).

The default C calling convention (`_cdecl`) is as follows:

- Arguments: passed through the stack, right to left
- Cleanup: the caller removes the parameters from the stack after the called function completes
- Return: `EAX`
- Caller-saved registers: `EAX`, `ECX`, `EDX` (the other are callee-saved)

Microsoft's `_stdcall` convention is slightly different:

- Arguments: as in `_cdecl`
- Cleanup: the callee is responsible for clearing the parameters before returning
 - To do this, the functions needs to know the right number of parameters passed. This means that this convention can only be used with fixed-length arglist functions (e.g. not `printf`)
- Return: as `_cdecl`
- Caller-saved registers: as `_cdecl`

Another convention is `_fastcall`:

- Arguments: Up to 2 parameters passed via registers `ECX` and `EDX`, the others are pushed to the stack as in the other
- Rest is the same

Linux's AMD64 convention (System V) works as follows:

- Arguments: passed in registers `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, the subsequent ones are on the stack (ordered like in the other conventions)
- Cleanup: done by caller
- Callee-saved registers: `rbx`, `rsp`, `rbp`, `r12`, `r13`, `r14`, and `r15`
- Caller-saved registers: `rax`, `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, `r10`, `r11`
- Return value: `rax` (`rax:rdx` if 128 bits are needed)

Access control

Access control is binary decision: we are allowed or not. At scale, however, we cannot enumerate all possible answers, we need **rules**. So we need to establish how we write these rules, how we express them in practice and how do we enforce them.

The **reference monitor is the agent that enforces the control policies**. All modern kernels have one implementation. The reference monitor needs to be :

1. **Tamper proof**
2. **Un-bypassable**
3. **Small enough to be verified/tested**

Access control encompasses **both authentication and authorization**:

1. **Authentication**: the reference monitor **needs to verify the identity of the principal (user) making an access request**
 - The user enters username and password. The login process creates a process that runs with access rights equal to those of the user
2. **Authorization**: the reference monitor **decides whether access is granted or denied**
 - The reference monitor has to find and evaluate the security policy relevant for the given request

The above tasks are easy in centralized systems but non-trivial in distributed systems.

To model the access methods, we use the **following concepts**:

- **User**: a person
- **Principal**: the user's identity (the name used in the system)
- **Subject**: the entity making a request within the system
- **Object**: what is being requested

- **Access operations:** all the operations executable on the requested object

Discretionary access control

Resources have a **owner that discretionarily decides its access privileges**. All common OSs and applications use DAC. The general model of DACs (**HRU model**) is the following:

- We need the following entities:
 - **Subjects** who can exercise privileges
 - **Objects** on which privileges are exercised
 - **Action** which can be exercised
- **Protections state:** a triple (S, O, A)
 - A : matrix with S rows and O columns
 - $A[s, o]$: privileges of subject s over o
- **Basic operations:** create/destroy subject/object or add/remove permissions
- **Transitions:** sequence of basic operations
 - **Transitions need to be applied atomically:** every steps is executed iff the previous succeed and if one fails they need to be rolled back
 - **Given an initial protection state and set of transitions, is there any sequence of transitions that leaks a certain right r (for which the owner is removed) into the access matrix? If not, the system is safe with respect to right r .**

The problem is **undecidable**; it becomes **decidable only if we disallow transitions**.

In real world OSs, we do not have an access matrix (if there were one, it would be sparse) but more optimal representations:

- **Authorization tables:** records non-null triples (typically used in DBMSs)
- **Access Control Lists:** records by columns (for each object, we store the list of subjects and authorizations)
 - Efficient with per-object operations
 - Most common case
 - Some systems can use abbreviated ACLs
 - Cannot have multiple owners
- **Capability Lists:** records by row (for each subject, we store the list of objects and authorizations)
 - Efficient with per-subject operations
 - Usually objects change and subjects stay
 - Capabilities are optional in POSIX OSs

Shortcomings of DACs:

1. **Cannot prove safety**
2. **Control access to objects but not to the data inside objects**
 - Malicious user attacks
 - Trojan horse attack: malicious program running with privileges of the user
3. **Problems of scalability and management**
 - Each user-owner can potentially compromise security of the system with their own decisions

Mandatory Access Control

Privileges are set by a **security administrator which defines a classifications of subjects (clearance) and of objects (sensitivity)**. The classification is composed of a **strictly ordered set of secrecy levels and a set of labels**.

Classification is a **partial order relationship that defines a lattice**. The order is imposed by the **dominance relationship**:

$$\{C_1, L_1\} \geq \{C_2, L_2\} \iff C_1 \geq C_2 \wedge L_2 \subseteq L_1$$

This means that we can have **some combinations that are not comparable between each other**.

The **Bell-LaPadula model** defines two rules:

1. **No read-up**: a subject s at a given secrecy level cannot read an object o at a higher secrecy level
2. **No write-down**: A subject s at a given secrecy level cannot write an object at a lower secrecy level
3. (For DACs): use of an access matrix to specify the discretionary access control

If we add the “**tranquility property**”, i.e the fact that the secrecy level of objects cannot change dynamically, **we have a secure system where information monotonically flows towards higher secrecy levels**. To make it possible for higher-ups to write public information **we need trusted subjects who can declassify or sanitize documents**.

The Bell-LaPadula model **does not address integrity**, we need **another model such as the Biba model** (basically just inverts the rules of the Bell-LaPadula).

Software security

Good software engineering ideally produces secure software. However, creating software is one thing, creating secure software is another and is very hard.

Safety is an implicit non-functional requirement of all software. An unmet security specification is a vulnerability (NOTE: vulnerability is not synonymous with an exploit).

Principles for writing secure programs:

1. **Reduce privileged parts to a minimum**
 - If possible, discard privileges definitely as soon as possible
2. **KISS**
3. **Do not rely on security through obscurity**
4. **Be wary of concurrency and race conditions**
5. **Fail-safe and Default-deny**
6. **Avoid the use of shared resources or unknown/untrusted libraries**
7. **Filter input/output**
8. **Do not write your own cryptographic primitives, password/secret management**
9. **Use trustworthy RNGs**

Binary exploitation

Buffer overflows

Assumptions: elfs on linux ≥ 2.6 on x86. The concepts, however, **apply to any kernel/architecture with the appropriate modifications**.

At every function call, the CPU executes the same (depending on calling convention) instructions:

```
...                ; register saving and parameter passing
call 0x8048484 <foo> ; equivalent to push eip && jmp 0x8048484
push ebp           ; save the current stack base onto the stack
mov  ebp, esp      ; the new base is the top of stack
sub  esp, 4         ; make room for function variables
...                ; function code
```

The same happens on function end:

```
...
leave ; Equivalent to: mov esp, ebp
;                pop ebp
```

```
ret    ; restore the saved eip
...    ; deallocation of saved registers / function arguments
```

Stack smashing is a well-known idea (see article by aleph1). Basically how it works is: **function allocates a buffer, buffer is filled *without bound-checking*, boom.**

Most common smelly-functions: strcpy, strcat, fgets, gets, sprintf, scanf.

To exploit this vulnerability, we can **pass specially formatted strings (via environment variables or other user-controlled means) such that we put the address that we want to jump into the place we want.** The **simplest way** is to jump into the buffer itself and **reuse the buffer we are smashing the stack with.**

A more elaborate way is to jump into a function that we know will be loaded and at which location > (e.g. libc functions).

The method described in this notes does not work on new OSs because all pages used for process stacks are marked as non-executable. To make this work we need to pass special flags during compilation.

nop sled

However, we **do not know the exact address of the buffer**: we only know that it is **somewhere around the esp**. We could **look it up using gdb**, however **different machines/executions can produce different results** (due to e.g. different invocation chains/environment variables etc...). Moreover, **debuggers often add offsets to the allocated process memory** (this means that the esp obtained with gdb is different to the effective one). This imprecision can **cause us to misread our buffer and to not execute the instructions we want**. Since our imprecision is only of a few bytes, we can **add a “landing strip” of nop instructions so that we can jump in the middle of this strip and still execute the correct code.**

Shellcode

What code do we put into our buffer? **Usually we want execve("/bin/sh")**. Since historically this has always been the case, the **payload of exploit is called shellcode**.

In Linux, a syscall invocation follows the following convention:

```
movl  eax, $syscall_number
mov   ebx, arg1           ; syscall arguments
...
int   0x80               ; Switch to kernel mode
```

To generate our exploit we can write a MW C program and disassemble it:

```
int main() {
    char *hack[2];
    hack[0] = "/bin/sh";
    hack[1] = NULL;
    execve(hack[0], &hack, &hack[1]);
}
```

We need to construct the following structure in memory to invoke execve:

```

      /-----\
      v         |
+-----+-----+
| /bin/sh | 0 | Address | NULL |
+-----+-----+

```

Is **pseudo-assembly** we have the following shellcode (everything is **parametrized in function of ADDRESS**).

```
movl  array-offset(ADDRESS), ADDRESS
movb  nullbyteoffset(ADDRESS), 0x0
movl  null-offset(ADDRESS), 0x0
```

```

;;
movl eax, $0xb
mov ebx, ADDRESS
leal ecx, array-offset(ADDRESS)
leal edx, null-offset(ADDRESS)
int 0x80

```

How do we get the exact address of '/bin/sh'? We can use the following **trick**: `call` pushes the return on the stack, this means that **executing a call just before declaring the string has the side-effect of leaving the address of the string on the stack.**

```

jmp offset-to-call
popl esi
;;
movl array-offset(esi), esi
movb nullbyteoffset(esi), 0x0
movl null-offset(esi), 0x0
;;
movl eax, $0xb
mov ebx, esi
leal ecx, array-offset(esi)
leal edx, null-offset(esi)
int 0x80
;; vvv exit(0) for cleanness vvv
movl eax, 0x1
movl ebx 0x0
int 0x80
;;
call offset-to-popl
.string "/bin/sh"

```

If we were to translate the real code into binary, **many of the instructions have \0 in them, meaning that they cause string operations to misbehave.** We simply need to **use shortened instructions and zeroed out registers** (via `xorl`) instead of `0x0`.

The remaining part is filling the buffer with nops and put the our guess of `esp` into the saved `eip`.

Using environment variables

Pros:

- **Easy** to implement (“unlimited” space for the string)
- **Easy** to target (we can know precisely the location)

Cons:

- Works for **local exploits** only
- The program **may wipe the environment**
- **Memory must be marked as executable**

Since the addresses will be precise, we might not need the `nop-sled`, however it adds a bit of reproducibility. **To activate the exploit we simply need to run slam the stack with our envvar’s address.**

Using built-in functions

Pros:

- Works **remotely** and **reliably**
- **No need for executable stack**
- A function is always executable

Cons:

- More **difficult to prepare** (need to craft the stack frame very carefully)

Basically what we need to do is **prepare the stack to trick the ret into jumping into system()**. The stack needs to be setup as if it has been called legitimately.

Other ways of jumping

1. Saved `eip` (**direct jump**) (what we done until now)
2. Function Pointer (**call another function**) (`jmp` into another function)
3. Saved `ebp` (**frame teleportation**)

Defending against buffer overflows

We have **three levels to try to block the exploitation** of this vulnerability:

1. **Source code level**: removes the vulnerability
 - Basically **write good code**:
 - Use safe functions `strncpy/strlcpy`
 - Do not use C, use a safer language
2. **Compiler level**: makes vulnerability non-exploitable
 - **Warnings** at compile-time
 - **Randomized reordering of stack variables**
 - Embedding **stack protection mechanisms** at compile time
 - Verifying, during the epilogue, that the **frame has not been tampered using a canary** inserted between local variables/control values that is checked (c.f.r gcc's StackGuard)
 - Canaries can be: random, all `\0` or random `xor` canaries (random `xor` canaries are the best protection)
3. **OS level**: makes exploitation harder
 - **Non-executable stack**
 - Bypassable (just don't put code into the stack)
 - Some programs require executable stack (e.g. `jvm`)
 - **Address space layout randomization**: repositions the stack at each execution at random

Format string bugs

Example of vulnerable code:

```
#include <stdio.h>

void test(char *arg) {
    char b[256];
    snprintf(b, 250, arg); // !!! "naked printf" vulnerability
                           // correct code should have been: snprintf(b, 250, "%s", b)
                           // NEVER use a naked variable as a format
    printf("buffer: %s\n", buf);
}

int main (int argc, char* argv[]) {
    test(argv[1]);
    return 0;
}

// ~> ./a.out "%x %x"
// buffer: b7ff0590 804849b    # We read random variables from stack and printed them on
//                               # stdout
```

The above code shows a simple way in which we can read everything we in memory: **using `%N$x` we can print the N-th integer of the function's arguments (in our case the N-th integer on the stack)**. This means that **we are not limited by the size of the format string and thanks to the power of scripting we can scan the whole stack** like this:

```
for i in $(seq 1 150); do
  echo -n "$i " && ./vuln "AAA %i\$x"
done
```

If we can scan the stack, we can **leak information** (data, addresses).

Fortunately for us, there is a placeholder that can **also write memory**: `%n` writes the number of characters printed so far at the address in the corresponding parameter. How can we abuse this?

1. **Put on the stack the address** of the memory (address) cell to modify
2. Use `%x (%N$x)` to **go find on the stack the address we want to modify** (let's call it `pos`)
3. Use `%n` instead of `%x` to **write a number in the cell pointed to by address**

To control the number we are writing we can use `%0Nc`: writes a character trying to fill a `N` character long string. An example format string that writes `n` could be:

```
<address>%<n-len(address)>c%<pos>$n
```

There is a problem in our method: `%c` can write only 2 bytes at a time. This means we need to do two writes to write a full 32-bit integer. Since we can only increment `n` (if we do not overflow obviously), we first need to write the word with the lower absolute value and then the other. Let us revise our procedure:

1. Put on the stack two addresses, `addr` and `addr + 2`
2. Use `$x` to go find the two find the first word `pos` (the second one will be `pos+1`)
3. Use `%c` and `%n` to write the lower absolute value in the cell pointed by `pos` and the other in `pos+1`

```
<addr><addr + 2>%<lower_val>c%<pos>$n<higher_val>c%<pos+1>c$n
```

To avoid possible side-effects when writing with `%n` (since it writes dwords and we are using it two times) we can use `%hn` (which writes words).

```
<addr><addr + 2>%<lower_val>c%<pos>$hn<difference between the two halves>c%<pos+1>c$hn
```

What to edit

We have **different options**:

1. The **saved EIP** (like in the buffer overflow case)
2. The Global Offset Table (**GOT**)
 - Dynamically relocate functions
3. **C library hooks**
4. **Exception handlers**
5. **Other** structures/pointers

Countermeasures

The **memory error countermeasures seen previously can help** against these attacks. Moreover, **modern compilers will warn** when they detect a naked `printf`. There also exist **patched versions of libc that mitigate** the problem (count-and-check and FormatGuard).

Conceptually, format string bugs are **not specific to formatting functions**. In theory, **any function with the same unique combination of characteristics is potentially affected**:

1. **Variadic function**
2. A **mechanism** (e.g. placeholders) to **indirectly read/write arbitrary locations**
3. The **ability of the user to control them**

Remember: `printf`-like functions interpret input that **has been proven to be Turing complete**, so we are basically injecting arbitrary code in memory.

Web applications

Web applications are built **on top of HTTP(S)**: a **plain-text**, **stateless** and **almost unauthenticated** protocol with **statefulness** and **authentication bolted on as extensions**. Crafting malicious requests is very easy.

The golden rule of web application security is that the **client is never trustworthy**. We need to **filter and check carefully anything** that is sent to us. The challenge is that with JavaScript, clients have become a (cooperative) part of the application.

How do we filter the input? It is not easy. The **sequence of validation/filtering** is:

1. **Allowlisting**: only allowing through what we expect
2. **Blocklisting**: on top of that discard known-bad stuff
3. **Escaping**: transform special characters into something else which is less dangerous

General rule: **allowlisting is safer than blocklisting**.

Cross-site scripting (XSS) and Same-Origin Policy

Cross site scripting is a **vulnerability** by means of which **client-side code can be injected in a page**. There are three types of XSS:

1. **Stored** (persistent) XSS: the **attacker's input is stored on the target server in a database**; then a victim retrieves the stored malicious code from the web application without that data being made safe to render in the browser
2. **Reflected** (non-persistent) XSS: (Attacker) **Client input is returned to the client (Victim) by the web application in a response**; the response includes some or all of the input provided in the request, without being stored and made safe to render in the browser.
3. **DOM-based XSS**: **user input never leaves the victim's browser**; the malicious payload is **directly executed by client-side script**

JavaScript is **sandboxed**, but **can also do some malicious things** like:

1. **Cookie theft** or **session hijack**
2. **Manipulation** of a **session** and execution of **fraudulent transaction**
3. **Snooping** on private information
4. **Drive by Download**
5. Effectively **bypass the same-origin policy**

The **same-origin policy** is implemented by all web clients and it **mandates that all client-side code load from origin A should only be able to access data from origin A**. An origin is defined as a <protocol, host, port> tuple. Modern web has **blurry boundaries** on this, like **CORS** and **client-side extensions**.

A gut reaction to prevent XSSs is to block the <script> tag (since we cannot know a-priori what users want to write), however that is not enough:

- <applet>, <frame> and <iframe> all execute code
- Even “safe” tags are not safe: we can inject JS using event handlers like `onerror` and `onload` or using `javascript::`

```

<svg onload="alert('XSS');" />
<a href="javascript:alert('XSS')">Totally safe link, click me!</a>
<a
  href="javas
cript:alert('XSS')"
>Totally safe link, click me!</a>
>
<a href="javasc&#09;ript:alert('XSS')">Totally safe link, click me!</a>
```

The potential blacklist becomes infinite. The solution is to escape text:

- < becomes <
- > becomes >
- & becomes &

So that it will *never* be interpreted as JS but still be rendered correctly.

Content Security Policy

It is a W3C specification to **inform the browser on what should be trusted and what not**. It creates an **allow list of safe-sites from which we can load code** relaxing and making SOP more flexible.

Technically, it's a **set of directives sent by the server to the client in the form of HTTP response headers**:

Content-Security-Policy: script-src 'self' http://necst.it

It is a **very effective remedy against XSS**, but **strict policies break functionality** while very **relaxed policies can be easily bypassed**. Moreover, the **policies need to be kept up to date manually** (most of the time) since pages and resources change. CSP can also break browser extensions that inject code into pages.

For the reasons above, it is **very rarely used**.

SQL injection

DON'T DO THIS:

```
SqlCommand cmd = new SqlCommand(String.Format(
    "SELECT * FROM Users WHERE username='{0}' AND password='{1}';",
    txtUser.Text, txtPassword.Text));
SqlDataReader reader = cmd.ExecuteReader();
```

Use **PreparedStatement** with placeholders and proper sanitization/permissions.

Injects can:

1. Be **done on single tables**
2. Be **done on different tables** with compatible columns using **union**
3. **Insert multiple data** with one insert
4. Be done **blindly**: we **don't need to necessarily retrieve data**, but we could exploit some other **characteristics of the response** (delay or errors) to infer the ability to do injections

Recap on code injection problems

The **root** of the problems outlined here is the **mixture between data with code** due to two conflicting requirements: the **functional requirement** wants them mixed while the **security one doesn't**.

The consequence is that **if at any point there is a parsing routine that reacts to some control sequences in the data, we have a vulnerability**.

Information leaks

Unintentional information leaks due to:

1. **Too detailed error messages**
 - Useful to have in **development** but should be turned off when in production
2. **Debug traces left active in production**
 - Can reveal **sensitive information** about client/server versions, DB names and structures etc...
3. **Insertion of user-supplied data in errors** (risk of reflected XSS)
4. **Side-channels** (e.g. "User not found" vs. "Password mismatch")

Password security

1. Passwords should **ALWAYS** be salted and hashed
2. **Password reset schemes** are **very sensitive** since it is **basically an alternate password**:
 - **DO**: Send private, temporary reset link to registered email
 - **DON'T**: send temp password, send the password (implies that you store it unhashed), ask only security questions

Bruteforcing protection

1. fail2ban to avoid bruteforcing passwords
2. Make accounts not enumerable (avoids reverse bruteforcing)
3. Blocking IPs is not a good idea (can be easily circumvented with proxies, VPNs etc.)

Cookies

Cookies provide **client-side information storage**. Original idea was to allow site personalizations, however they **become dangerous if we use them to keep track of user authentication and sessions**.

Session cookies need to be totally **random and not guessable**. They **may be encrypted (+MAC)** if they contain sensible information (not needed anymore due to TLS). **Any token (not only session) need to have a reasonable expiration date**.

Since server-side sessions require storage space, we can **send all the needed variables to the client, signing them with a server-side private key** (basically encrypted JWTs).

Session pose some **challenges**:

1. **Concurrency**: what if we have a replicated server and a user logs into one of them
2. **Session termination**: when/how do we terminate a session?
3. **Storage**: on disk or in RAM (do not). Saving to secondary storage has performance implications, even in a replicated case (see concurrency)

Session hijacking is somehow obtaining a session (XSS or bruteforcing session IDs).

Cross-site request forgery (CSRF)

It is an attack that **forces an user to execute unwanted actions (state-changing) on a web application in which they are currently authenticated with ambient credentials** (credentials sent automatically every request, i.e. cookies).

Key concepts:

- All requests originating from the user's browser come with the user's cookie jar
- Malicious requests are routed to the vulnerable web application through the victim's browser (via e.g. a redirect or a crafted request)
- Websites cannot distinguish if requests are coming from an authenticated user have originated by an explicit user action

CSRF tokens are a remedy: they are a **random challenge token unique to each user session that is regenerated at each request and has an expiration**. The client sends it to the server, which then checks that it has the value it expected. The token is **not stored in cookies**.

Another mitigation is trying to **not send cookies when going from the malicious link to our website**. In the modern web, we can use the **SameSite attribute** to specify this behaviour:

- **SameSite=strict**: don't send the cookie for **any cross-site usage**
- **SameSite=lax**: send cookies only for **cross-domain navigation**

Network protocol attacks

We will see three typical types of attacks:

1. **Denial of Service:** attack on availability
2. **Sniffing:** attack on confidentiality
3. **Spoofing:** attack on integrity and authenticity

DoS

It is not solvable. Some types+examples are:

1. **Killer packets:** one malformed packet and we crash the kernel
 - **Killer ping:** ICMP echo request have a maximum size (65535). If we create a bigger buffer (like 65527), we may cause a buffer-overflow in the server's kernel and and panic.
 - **Teardrop:** exploits vulnerabilities in TCP reassembly (we send packets with overlapping offsets); this can cause kernel hangs/crashes.
 - **Land attack:** a TCP packet with `src IP == dst IP` and a SYN flag set could loop the kernel and lock up the TCP/IP stack.
2. **SYN flood:** flooding does DoS by wasting a server's memory
 - The servers needs to keep track to each SYN packet it receives since it needs to ways for an ACK from the initiator. If we generate a lot of SYN, we saturate the server's memory, making it impossible to open new connections (note the asymmetry of resource usage between the two parties).
 - **A remedy to this are SYN-cookies** (usually cryptographically hashed sequence numbers). It offloads some memory usage to the client since the server opens a connection only is it receives a valid the SYN-cookie in the last ACK message. This avoids filling memory with half opened connections
3. **DDoS:** like a DoS, but distributed among multiple different attackers (more difficult to detect)
 - **Botnet:** a **network of compromised computers** (called bots) that are **connected to a command-and-control infrastructure** that allows the attacker (botmaster) to send commands to the bots
 - Botnets are not only used for DDosing, but also other malicious attacks (spamming, phishing, information stealing etc.)
 - **Smurfing:** the **attacker is outside the LAN** that connects a lot of hosts to a target; the **attacker can send a spoofed ICMP ping with the target's IP in broadcast**, meaning that **all hosts in the LAN will reply to the victim**
 - The phenomenon of creating a lot of requests from a single request is called **amplification** (of bandwidth or of packet size)
 - ICMP is not the only protocol that allows amplification
 - This attack is **not possible anymore:**
 - * Block broadcast requests or block them for hosts outside the LAN
 - * Disable the abusable protocols (e.g. ICMP)

Sniffing

Normally, a **NIC intercepts and passes to the OS only the packets directed to the host's address. However, we can configure it to pass ALL packets to the OS (promiscuous mode).**

To **mitigate** sniffing is using **switched networks** instead of hub-based networks, since switches selectively relay traffic while hubs broadcast it.

Spoofing

ARP Spoofing (or Cache poisoning)

The ARP cache maps 32-bits IPv4 addresses to 48-bits MAC addresses. Each host **caches the ARP replies and can only trust its entries**. Entries are **refreshed every ~30 seconds** due to changes in the network with broadcast messages on the LAN.

This cache works in a “**first-come first-trusted**” way and **lacks authentication**: this means that an attacker can **forge replies and if it wins the race** with the real host, we can **spoof the real MAC address and redirect traffic**.

Some **mitigations**:

1. **Check responses before trusting**
2. Adding a SEQ number, or other kinds of **unknowns** for each request

Filling the CAM tables

Switches use **CAM tables** to know which **MAC addresses** are on which **ports**. If this table **gets full**, the **switch cannot cache any more addresses** and **resorts to forwarding everything** (MAC flooding).

We can also **abuse the Spanning Tree Protocol**: switches decide how to build the ST by exchanging **BPDUs** (bridge protocol data unit) packets to elect the root node, which are **obviously not authenticated**. This means that **an attacker can change the shape of the tree for sniffing/spoofing purposes**.

IP address spoofing

The **IP source address is not authenticated**. Changing it in **UDP or ICMP packets is easy**. However, the **attacker will not see the answers** (e.g., they are on a different network), because they will be sent to the spoofed host (blind spoofing). But **if the attacker is on the same network**, they can sniff the rest, or use **ARP spoofing**.

For **TCP** things are **more complicated**. We need to **guess correctly guess a SEQ number to hijack a connection**. Initial sequence numbers (**ISN**) should, however, be random.

If we **spoof the source IP** and **initiate a connection**, we must be sure that the **spoofed machine does not receive the SYN-ACK**, otherwise it would **RST and break everything**. To avoid this, we could **DoS the spoofed machine**, to make it impossible to respond to packets, and guess the starting **ACK** number of the server.

TCP session hijacking works similarly to what is outlined above, the **difference is that the attacker sniffs both SEQ and ACK before disrupting the original connection** and putting himself in the middle.

The attacker can **avoid disrupting the spoofed connection** and just inject things in the flow **only if he is a man in the middle**, i.e. if he can control/resync all the traffic flowing through.

Man In The Middle (MITM)

It is a broad category of attacks where an **attacker can impersonate the server with respect to the client and vice-versa**. It can be:

1. Physical or logical
2. Full or half-duplex

DNS cache poisoning

Like ARP caches, **non-authoritative DNS servers store the DNS records in a first-arrives-first-trusted fashion**. This means that by:

1. Sending a DNS query to the server to attack makes it ask an authoritative server for a resolution
2. **Being faster than the authoritative server** to respond

We can **poison the DNS** record making anyone who uses the DNS server receive a malicious IP.

The above version of the poisoning attack **has been fixed thanks to some weak authentication**, however we can **still do it by slightly changing the way** we do it:

1. The attacker makes a recursive query to the victim DNS server
2. The victim (non authoritative) DNS server contacts the authoritative server
3. The attacker, impersonating the authoritative DNS server, spoofs the answer (before the legitimate one)
4. The victim DNS server trusts and caches the malicious record

DHCP poisoning

DHCP is unauthenticated. The attacker can then intercept the DHCP requests and be the first to answer. The client will trust the first to respond.

With a spoofed DHCP response the attacker can set IP addresses, DNS addresses and the default gateway of the victim client.

ICMP and ICMP redirects

ICMP is a control protocol used by IP. Some interesting commands are:

1. Echo: pinging
2. Redirect: sent by routers, informs hosts about better routes/gateways
 - This feature is not used any more and every system will have ICMP redirect turned off

The attacker can forge a spoofed ICMP redirect to re-route traffic. ICMP includes some weak authentication: it includes the IP header and a portion of the payload of the original IP datagram to prove that it received the message. Of course this doesn't stop attackers since they can just sniff packets and try to win the race condition with the router.

Route mangling

If the attacker can announce routes to a router, they can play a number of magical tricks since:

- IGRP, RIP, OSPF: no/weak authentication
- EIGRP, BGP: authentication available but seldom used

Secure network architectures

Firewalls

They are network access control system that verifies all the packets flowing through it. Their main functions are usually IP packet filtering and Network Address Translation (NAT).

A firewall checks **all** the traffic flowing through it, and **only** the traffic flowing through it. This means that they are powerless against local-network attacks or unchecked path attacks. The firewall itself is a computer: it could have vulnerabilities and be violated. Most of the times it is a single purpose machine, an embedded appliance with just a firmware, and offer few or no services (to reduce attack surface).

A firewall acts like a bouncer: it just applies some rules to packets enter/leaving through it. This means that bad policies lead to bad security. Policies must be built on a default-deny basis. There are two main types of firewalls:

1. Network layer firewalls:
 - Packet-filters: looks up to the IP layer (and partially transport)
 - Stateful packet-filters: can track state and manage protocols such as TCP
2. Application layer firewalls:
 - Circuit level firewalls (legacy): basically TCP-level proxies, not transparent to users
 - Application proxies: normal proxies, not transparent to users

Packet filters

Every packet filter allows us to express the following concept: "if packet matches certain condition, block/allow/log/etc...".

Stateless packet-filters do packet-by-packet processing and cannot track the state of connections. They do not inspect the payload. In some configurations, it may require relaxing policies to let through some connections:

- For example, in a web-server, we would need to allow traffic in both directions (from outside to server and viceversa), meaning we can be allow leakage of info from the local network to the outside

Stateful packet-filters do everything that stateless packet filters do, plus they implement the TCP state machine. This means that we can track connection without adding a “response” rule, making deny-rules safer:

- Using the same example as before, we could deny packets from the local network while still allowing connections from the outside to work

The main problem of stateful packet filtering is performance: performance is bounded on a per-connection basis, not on a per-packet basis. This means that the firewall is more vulnerable to DoS.

Stateful packet filtering allows some more advanced techniques:

1. Tracking (logging/accounting)
2. Deep content inspection: we are able to reconstruct application-layer protocols and inspect them
 - This means that we can instruct the firewall to analyze packets and recognize attacks
3. Packet defragmentation and reassembly
4. NAT:
 - for TCP, see first-year FCI lecture on NAT
 - in UDP, we do not have connections meaning we need to infer connections based on context with something like timeouts (abusable and not always reliable)
 - Some protocols transmit network information at application layer, requiring application layer inspection for NAT to work:
 - FTP is particularly a pain to deal with since it does things kinda backwards (client sends a free port to the server and then the server initiates a connection)

Application proxies

Proxies act as servers to clients and clients to servers (basically a MITM). They are almost never transparent to clients, which may require modifications. Moreover, each application protocol requires its own proxies.

Proxies can do a bunch of useful stuff:

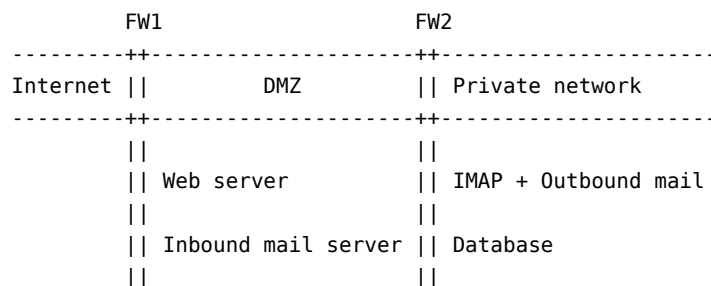
1. Authenticate users
2. Apply specific filtering policies
3. Content filtering/scanning
4. Expose a subset of the protocol to defend clients or servers (if a server is being defended we call them reverse-proxies).

Architecture of secure networks

Multi-zoned networks

The firewall model is called “castle-model”: we build a wall around us to block all traffic. However, we cannot completely isolate ourselves from the outside (basically just cut the internet cable) and we cannot assume that what is inside is “good” and what is outside is “bad” (e.g. remote login to internal network or web servers hosted in the internal network that should be available to all users). If we mix externally accessible servers with internal clients, we lower the overall security (need to punch holes in the castle walls); the solution is to split the network by privilege levels and use firewalls to regulate access.

In practice, we create a semi-public zone called DMZ (demilitarized zone) that will host public servers but NOT critical or irreplaceable data. The DMZ is almost as risky as the internet.

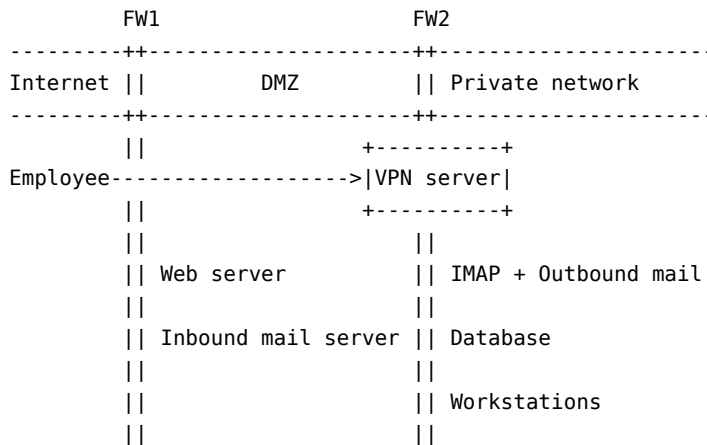




The above diagram kinda explain why firewalls are called firewalls: they do not stop attackers (once we get access to the local network we can do whatever (spoof and sniff packets)), but they contain a break-in for just enough for us to prepare a defense.

VPNs

How can we allow authenticated access to internal services form the outside (e.g. remote login for employees)? The solution is to use a VPN, which is an encrypted overlay connection over a public network.



VPNs can work in two modes:

1. Full tunnelling: Every packet goes through the VPN tunnel
 - Multiplies traffic, could be inefficient
 - Single point of control and application of all security policies as if the client were in the corporate network
2. Split tunnelling: Traffic to the internal network goes through the tunnel, everything else goes through the ISP as normal
 - More efficient, less to control
 - Firewall is not the only point of contact between the internal network and the internet

Some VPN protocols:

1. PPTP: proprietary Microsoft protocol, variant of PPP with cryptography and authentication
2. IPSEC: security extension of IPv6 backported to IPv4, very flexible but very complicated
3. VPN over TLS: most used (see OpenVPN for a open source implementation)
4. SSH tunnelling

SSL, TLS and SET

SSL/TLS

HTTP over SSL (HTTPS) was the initial usage of SSL (developed by Netscape). It provides communication confidentiality and integrity, mutual authentication but no guarantees on data usage and no strict authentication of the client. SET was an alternative that offered guarantees on data usage and transaction security enforcement, however it never gained traction. SSL would eventually be standardized by the IETF as TLS after SSL version 3. Today we are at TLS 1.3, versions of TLS up to 1.1 are insecure and should be avoided.

TLS enforces confidentiality and integrity of the communications, server authentication and optionally client authentication too. It uses both symmetric and asymmetric cryptography for performance reasons.

Everything starts with the TLS handshake:

1. The client starts the connection with the server by sending a cipher suite plus some random data

- The cipher suites (one for each party, can be different; the standard mandates a minimal cipher set) are compared during handshake to agree on a shared algorithm; it is composed of :
 - Key exchange/encapsulation algorithm
 - Symmetric encryption algorithm
 - Digital signature algorithm
 - Hash function
- 2. The server responds with:
 - The cipher selection
 - Some other random data
 - A server certificate (which the public key and the identity of the server)
- 3. The client checks the server certificate (validity, trusted root CA, name of the server)
- 4. If the certificate is ok, the client generates a pre-master secret, encrypts it with the servers public key and sends it
 - If we want client authentication, we need to sign this secret with the clients private key and also send the client certificate
- 5. Both parties compute the shared secret by hashing the pre-master secret, the client random data and server random data
- 6. The communication then proceeds on an symmetrically-encrypted channel

What can a MITM do to break it:

1. Sniff: the MITM can see the traffic, but cannot decrypt it since it lacks the pre-master secret
2. Intercept the original certificate and send a spoofed one:
 - Sending a fake certificate (i.e. signed by a non-root CA) will be rejected by the client
 - Sending a good certificate with a fake name will be rejected by the client
 - Sending a good certificate with a modified public key (making it invalid) will be rejected by the client

TLS is by design resistant to MITM... if the client behaves correctly. If the client can be fooled into trusting the invalid certificate everything is moot. This means that complimentary to TLS, we need good user interfaces that correctly communicate the dangers of trusting forged certificates. Moreover, TLS:

1. Does not provide protection before or after transmission (on server, on client or by an abuser)
2. Relies on PKI (if a certificate violates the trust everything is moot)
 - HSTS (HTTP Strict Transport Security): a header to tell the browser to always connect to that domain using HTTPS; browsers often implement a HSTS preload list; defends against SSL stripping
 - HPKP (HTTP Public Key Pinning): Deprecated, a header to tell the browser to “pin” a specific public key hashes for that origin for a period of time, if the origin does not use one of the pinned key, an error is shown; defends against trust cert mis-issuances
 - Certificate Transparency: when a CA issues a certificate, it lists its metadata to a publicly available log; this can help against certificate misissue

SET

Secure Electronic Transaction, was a joint effort by VISA+MasterCard consortium. It protects transactions and not connections. The high level approach is:

1. The card holder sends the merchant only the order of goods
2. Send the payment data to a payment gateway
3. Empower the gateway to verify correspondence

It uses the concept of a dual signature, i.e. a signature that joins together the two pieces of a message, directed to two distinct recipients.

```

+-----+ H +-----+
| Order info | ----> | Hash 1 | \           Encrypt with
+-----+          +-----+ \   H       holder info
                        | ----> Dual hash --\
+-----+ H +-----+ /           |
| Payment info | --> | Hash 2 | /           v
+-----+          +-----+           Dual signature

```

Using the above names, we have that the data flow is the following:

```

+-----+-----+
|           | Hash 2 | Enc. with merchant pub key | +---+
| Order info +-----+ =====> | M |
|           | Dual sgn |           | e |
+-----+-----+           | r |
                               | c |
                               | h |
+-----+-----+           | a |
|           | Hash 1 | Enc. with payment gate. key | | n |
| Pay info  +-----+ =====> | t |
|           | Dual sgn |           |
+-----+-----+           +---+
                               ||
++-----++      Payment info still encrypted  //
|| Payment gateway || <=====
+-----++

```

The merchant can verify the correctness of the information by checking the double signature:

1. Calculate the hash of the order info
2. Calculate the dual hash
3. Check that it corresponds with the received hash

The payment gateway can do the dual verification.

SET failed since it does require that each of the three parties have a certificate. For the payment gateway and the merchant is ok, but giving a certificate for each card holder does not scale at all (it requires pre-registration, making it less convenient meaning that people will not bother and use the less secure method).

Nowadays the problem is solved with a simple redirect with a token to the bank's site, or with 3-D secure.

Malicious software

Malicious software, or malware, is code that is intentionally written to violate a security policy. There are different "categories"/features of malware, some are:

1. Viruses: code that self-propagates by infecting other files, usually executables; it is not a standalone program
2. Worms: standalone programs that self-propagates, even remotely, often by exploiting vulnerabilities/social engineering
3. Trojan horse: apparently benign programs that hide functionality and allow remote control

Initially malware was created for demonstration purposes (research basically), around the 2000s the main driving factor shifted to monetization (ransomware to hit the mass population). After the 2010s we slowly seen the rise of viruses that target high profile targets/infrastructure created by organized crime groups, sometimes sponsored by state and not only driven by monetary incentives.

Viruses

Fred Cohen ('83), theorized the existence and produced the first examples of viruses. It is impossible to build the perfect virus detector (propagation detector):

- Let P be a perfect detection program
- Let V be a virus that uses P as a subroutine:
 - if P(V) = True; then halt; implies that V doesn't spread, thus it is not a virus
 - if P(V) = False; then spread implies that V spreads and is a virus
 - We have a contradiction, thus P is not a perfect detection program

Thus we can only create programs that detect a specific virus.

The general life cycle of malware is the following:

1. Reproduce: most modern does not self-propagate, however the choice of the infection vector is still relevant to the reproduction of the malware
2. Infect: we need to balance infection and detection possibility
 - Boot viruses: stored in the MBR of hard drives, quite old tech but regaining traction
 - File infectors:
 - Overwrite original program with virus
 - Parasitic virus (append code and modify program's entry point)
 - multi-cavity virus (inject code into unused parts of the program's code)
3. Stay hidden
4. Run payload

“Macro” viruses are the most common virus today. They are data files traditionally considered safe but with “macro” functionality that allows arbitrary code execution. They are very difficult to deal with and remove, especially in a corporate environment since they often infect documents stored on shared drives.

Worms

Today worms propagate via emails/social networks by emailing/messaging to others in the infected computer's address book. They usually attach files with executables or executables masquerading as data.

The modern version of the original idea of worms are called mass scanners. The basic pattern is the same: infect a computer and seek out new targets. They spread very fast to large number of hosts. They employ scanning techniques to choose the next target:

- Trying random addresses (with the rise in internet usage it became more viable)
- Local preference: do more scans to local networks since they usually are faster than the wide internet
- Permutation scanning: divide the address space between machines to spread more efficiently
- Hit-list scanning: prepare a list of vulnerable machines in the work to lift-off faster
- Warhol worm: hit-list + permutation, created to demonstrate that it can infect the whole internet in less than 15 minutes

Since 2004, however, there has been a silence: no more worm outbreaks have been seen. Moreover, no one has ever tried to take down the whole internet, even though there have been opportunities to do so. The reason for the lack of attacks to the basic infrastructure is a lack of motivation: the attackers need internet to spread the worm! The growing interest in monetization also put worms out of favour: the best way to monetize is not a hyper-spreading worm, but slowly propagate malware and make it as persistent/undetected as possible to gather info/sell as a botnet/run a crypto miner.

Botnets

A botnet is a network that consists of several malicious bots that are controlled by a commander, commonly known as botmaster (botherder).

Bots do not self-replicate and are persistent on the computer. They are a threat for:

1. The infected host: information harvesting
2. The rest of the internet: spam, DDoS, propagation or support infra for organized crime

Anti-malware

We can defend against malware by:

1. Patching vulnerabilities: most worms exploit known vulnerabilities, useless against zero-days
2. Signatures: must be developed automatically since worms operate faster than human response
3. Intrusion/anomaly detection

Antiviruses/Anti-malware software can use different methods to detect malware:

1. Signature-based detection (seen previously): requires a database of byte-level/instruction-level signatures that match known malware (wildcards are often used)
 - Basically a blocklist, thus somewhat ineffective

2. Heuristics: useful against worms and older viruses, checks for signs of infections for example
 - Code execution starts in last section
 - Incorrect header size in PE header
 - Suspicious code section name
 - Patched import address table
3. Behavioural detection: detect the common behaviour of known malware

Anti-malware software is not the solution to everything: if your only tool is a hammer every problem is a nail.

Virus stealth techniques

Virus scanners quickly discover viruses by searching around entry point. This means we need entry-point obfuscation (via multicavity, or by hijacking control later in the program).

Some stealth techniques used by all malware that self-replicates is:

1. Polymorphism: change the layout/shape of every infection
 - A way is to encrypt the payload with a different key for each infection
2. Metamorphism: create different “versions” of code that look different but do the same things

Some general stealth techniques are:

1. Having a dormant period
2. Having a event-triggered payload (like a C&C channel for botnets)
3. Encryption/packing (similar to polymorphism but more advanced)
 - Uses a small encryption routine that changes the key at each execution
4. Anti-virtualization:
 - If a program is not run natively on a machine, chances are high that it is being analyzed/scanned or debugged
 - Modern malware detects the execution environment to complicate analysis
 - Virtual machines are easily detected
 - Hardware supported virtual machine are still easily detected
 - Theoretically emulators are undetectable, however practically emulators are not perfect and these imperfection can be detected
5. Rootkits: once we become root, we plant a kit to remain root (thus root-kit) like making the files/processes/users/directories disappear or making the attacker invisible
 - Can be either in userland or kernel-space:
 - Userland: trojanized system-administration programs or backdoored login
 - * easier to build but easier to detect
 - Kernel-space: more difficult to build, but can hide anything effectively
 - * One of the main methods is syscall-hijacking, which basically consists in altering the SYS_CALL/Interrupt/Global descriptor tables and inserting trojanized versions
 - More complicated like BIOS-based, firmware based or embedded in virtualization systems
 - Rootkits are very difficult to detect, the main methods are:
 - A sysadmin that notices some discrepancies in the system’s behaviour
 - A trusted computing base (e.g. Secure Boot)
 - Cross-layer examination: examining the same element at different layers of the stack (e.g. comparing listing a directory with `ls` with physically reading the filesystem from the disk with the help of clean computer)